

OpenMP Exam Prep

Only the 5 Directives From Miss's Slides — Question, Answer & Simple Step-by-Step Explanation

Key Directives (from slides)

#pragma omp parallel	Creates a team of threads that run the block together
#pragma omp parallel for	Splits a for-loop's iterations across threads
private(var)	Each thread gets its own private copy of a variable
reduction(+:sum)	Safely combines a variable (e.g. sum) across all threads
omp_get_thread_num()	Returns the calling thread's ID number

The Golden Rule

Every question below uses ONLY these 5 directives. The teacher may change the logic (sum becomes factorial, becomes product, becomes even/odd count...) but the skeleton never changes: `#pragma omp parallel for` splits the loop, and `reduction(op:var)` safely combines the result. Master ONE pattern and you can answer all of them.

What's inside

1. Hello World — basic parallel region
2. Print thread ID (private + `omp_get_thread_num`)
3. Parallel for loop (splitting iterations, no reduction)
4. Array sum using reduction (Miss's original example)
5. Array product using reduction
6. Factorial of a number using reduction
7. Find maximum in array using reduction
8. Find minimum in array using reduction
9. Count even and odd numbers using reduction
10. Sum of squares from 1 to N using reduction
11. Sum of only the odd numbers in an array using reduction
12. Count how many elements are greater than a given value
13. Average of array elements using reduction
14. Sum of cubes from 1 to N using reduction

Q1. Write a simple OpenMP program that prints a message from multiple threads.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello from thread\n");
    }
    return 0;
}
```

Sample Output

```
Hello from thread
Hello from thread
Hello from thread
Hello from thread
```

Explanation — Step by Step (Simple Words)

1. Include `<omp.h>` — this gives you all OpenMP functions and directives.
2. `#pragma omp parallel` tells the compiler: 'run the block right after this on multiple threads at the same time', creating a **team of threads**.
3. The `{ }` block is the parallel region — every thread runs its own copy of everything inside it.
4. If the system has 4 cores, OpenMP typically creates 4 threads, so `printf` runs 4 times.
5. The order the lines print in is NOT guaranteed — threads can finish in any order.

Note: This is the base template every other question builds on.

Q2. Modify the program so each thread prints its own thread ID.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int id;
    #pragma omp parallel private(id)
    {
        id = omp_get_thread_num();
        printf("Hello from thread %d\n", id);
    }
    return 0;
}

```

Sample Output

```

Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3

```

Explanation — Step by Step (Simple Words)

1. `omp_get_thread_num( )` returns the ID of the thread currently running it (0, 1, 2, 3, ...).
2. `private(id)` gives each thread its OWN separate copy of `id`, so threads never overwrite each other's value.
3. Without `private(id)`, all threads would share ONE `id` variable and the printed numbers could get mixed up.
4. Thread 0 is always the original/master thread; 1, 2, 3... are the extra worker threads OpenMP created.

Note: Any time a variable must hold a DIFFERENT value per thread, make it private.

Q3. Parallelize a simple for loop so its iterations run on different threads.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel for
    for (int i = 0; i < 10; i++) {
        printf("i = %d\n", i);
    }
    return 0;
}

```

Sample Output

```
i = 0
i = 4
i = 1
i = 5
i = 2
...
(exact order may vary each run)
```

Explanation — Step by Step (Simple Words)

1. `#pragma omp parallel for` does two things at once: creates a team of threads, and automatically splits the loop's iterations among them.
2. Example: with 10 iterations and 4 threads, roughly thread 0 gets `i=0,1,2` ; thread 1 gets `i=3,4,5` ; and so on — OpenMP decides the exact split.
3. The loop variable `i` is automatically made private for you — you do not need to write `private(i)` yourself for the loop counter.
4. Because iterations run at the same time on different cores, the printed order is scrambled — this is expected.

Note: This is the base 'split the loop' skeleton — Q4 onward just adds reduction on top of it.

Q4. Write a program to calculate the sum of all elements of an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i, sum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};

    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < 10; i++) {
        sum = sum + arr[i];
    }

    printf("Total Sum = %d\n", sum);
    return 0;
}
```

Sample Output

```
Total Sum = 55
```

Explanation — Step by Step (Simple Words)

1. The array holds 1 to 10. Expected sum = $1+2+\dots+10 = 55$, so you can double-check easily.
2. `reduction(+:sum)` is the key part: each thread keeps its OWN private mini-total of sum, adds up its own share of the array, and at the end OpenMP ADDS all the mini-totals together into the final sum.

3. Why not just make sum shared? Because if 4 threads update the same shared sum at the exact same time, some additions get LOST — this is a race condition. reduction avoids this automatically.
4. The + in reduction(+:sum) is the operator being reduced — this same slot can also hold *, max, or min (see the next questions).
5. After the loop, sum already holds the fully combined, correct total.

Note: This is Miss's exact slide example — memorize this pattern first, everything else is a small variation of it.

Q5. Modify the sum program to instead calculate the **PRODUCT** of all array elements.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i;
    long product = 1;
    int arr[5] = {1,2,3,4,5};

    #pragma omp parallel for reduction(*:product)
    for (i = 0; i < 5; i++) {
        product = product * arr[i];
    }

    printf("Total Product = %ld\n", product);
    return 0;
}
```

Sample Output

```
Total Product = 120
```

Explanation — Step by Step (Simple Words)

1. Only two things changed from the sum example: reduction operator is now * instead of +, and the loop body multiplies instead of adds.
2. product starts at 1 (not 0!) — multiplying by 0 would wipe everything out. Sums start at 0, products start at 1. This is the 'identity value' rule.
3. Each thread multiplies its own chunk of the array into its private copy of product, then OpenMP multiplies all the private copies together at the end.
4. $1 \times 2 \times 3 \times 4 \times 5 = 120$, matching the printed result.

Note: Proves the golden rule: same skeleton as Q4, only the operator and starting value changed.

Q6. Write an OpenMP program to calculate the factorial of a number N.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n = 6;
    long fact = 1;

    #pragma omp parallel for reduction(*:fact)
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }

    printf("Factorial of %d = %ld\n", n, fact);
    return 0;
}
```

Sample Output

```
Factorial of 6 = 720
```

Explanation — Step by Step (Simple Words)

1. Factorial of N means $N \times (N-1) \times \dots \times 1$. Here $N=6$, so $6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$.
2. This is the SAME pattern as Q5 (product reduction) — just looping from 1 to n instead of looping over array elements.
3. `reduction(*:fact)` again gives every thread a private safe copy of fact so multiplications from different threads don't clash.
4. fact starts at 1, same identity-value rule as Q5.

*Note: If the teacher asks for factorial instead of sum, this is exactly the swap: + becomes *, array loop becomes 1..n loop.*

Q7. Find the MAXIMUM value in an array using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int arr[8] = {12, 45, 3, 67, 21, 9, 88, 34};
    int max_val = arr[0];

    #pragma omp parallel for reduction(max:max_val)
    for (int i = 0; i < 8; i++) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }

    printf("Maximum value = %d\n", max_val);
    return 0;
}

```

Sample Output

```
Maximum value = 88
```

Explanation — Step by Step (Simple Words)

1. `max_val` starts at `arr[0]` so it already holds a valid value from the array.
2. `reduction(max:max_val)` gives each thread its own private 'best value found so far' from its own chunk of the array.
3. Inside the loop, each thread just checks: 'is this element bigger than my current best? If yes, update it.'
4. At the end, OpenMP compares all threads' private best values and keeps the overall largest one.

Note: Works the same way for `reduction(min:...)` in Q8 — just flip '`>`' to '`<`'.

Q8. Find the MINIMUM value in an array using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int arr[8] = {12, 45, 3, 67, 21, 9, 88, 34};
    int min_val = arr[0];

    #pragma omp parallel for reduction(min:min_val)
    for (int i = 0; i < 8; i++) {
        if (arr[i] < min_val) {
            min_val = arr[i];
        }
    }

    printf("Minimum value = %d\n", min_val);
    return 0;
}

```

Sample Output

```
Minimum value = 3
```

Explanation — Step by Step (Simple Words)

1. Mirror image of Q7 — everything identical except reduction changes from max to min, and the comparison flips from '>' to '<'.
2. `min_val` starts at `arr[0]` as a safe starting guess.
3. Each thread privately keeps its own smallest-value-found-so-far.
4. OpenMP combines all threads' private smallest values and keeps the overall smallest as the answer.

Note: Q7 and Q8 prove it again: memorize ONE pattern, swap one word to answer both.

Q9. Count how many even numbers and how many odd numbers are in an array.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int evenCount = 0, oddCount = 0;

    #pragma omp parallel for reduction(+:evenCount, oddCount)
    for (int i = 0; i < 10; i++) {
        if (arr[i] % 2 == 0)
            evenCount++;
        else
            oddCount++;
    }

    printf("Even count = %d\n", evenCount);
    printf("Odd count = %d\n", oddCount);
    return 0;
}
```

Sample Output

```
Even count = 5
Odd count = 5
```

Explanation — Step by Step (Simple Words)

1. You can reduce MORE THAN ONE variable at once — just separate them with a comma inside the same `reduction(...)` clause.
2. Each thread keeps its own private `evenCount` and `oddCount`, increasing whichever one matches the number it's checking (`arr[i] % 2 == 0` means even).

3. At the end, OpenMP adds together all threads' private evenCount values into one final evenCount, and the same for oddCount.
4. Out of 1 to 10 there are exactly 5 even (2,4,6,8,10) and 5 odd (1,3,5,7,9) numbers.

Note: Common exam twist: 'count elements above a value', 'count negatives' — same reduction(+:count) idea, see Q12.

Q10. Calculate the sum of squares of numbers from 1 to N using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n = 5;
    long sumSquares = 0;

    #pragma omp parallel for reduction(+:sumSquares)
    for (int i = 1; i <= n; i++) {
        sumSquares += i * i;
    }

    printf("Sum of squares 1..%d = %ld\n", n, sumSquares);
    return 0;
}
```

Sample Output

```
Sum of squares 1..5 = 55
```

Explanation — Step by Step (Simple Words)

1. $1^2+2^2+3^2+4^2+5^2 = 1+4+9+16+25 = 55$ — matches the printed result.
2. Same sum-reduction skeleton as Q4, just with $i*i$ inside the loop instead of $arr[i]$, and looping directly over numbers 1..n instead of reading from an array.
3. `reduction(+:sumSquares)` keeps a safe private running total per thread, then adds them all together at the end.
4. Confirms the golden rule again: same pragma, only the math inside the loop body changes.

Note: Any 'sum of ... from 1 to N' question (cubes, odd numbers, etc.) follows this identical pattern.

Q11. Calculate the sum of only the ODD numbers in an array using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int i, oddSum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};

    #pragma omp parallel for reduction(+:oddSum)
    for (i = 0; i < 10; i++) {
        if (arr[i] % 2 != 0) {
            oddSum = oddSum + arr[i];
        }
    }

    printf("Sum of odd numbers = %d\n", oddSum);
    return 0;
}

```

Sample Output

```
Sum of odd numbers = 25
```

Explanation — Step by Step (Simple Words)

1. $1+3+5+7+9 = 25$ — only the odd-indexed VALUES are added, not all ten numbers.
2. This is Q4's sum pattern with one small addition: an if-check inside the loop before adding to the reduction variable.
3. Each thread still privately totals oddSum for its own chunk of the array, only adding when $\text{arr}[i] \% 2 \neq 0$ (meaning the number is odd).
4. OpenMP combines all the private oddSum totals at the end into the final answer, exactly like the plain sum example.

Note: Shows that reduction still works perfectly even when the loop body has an if-condition inside it.

Q12. Count how many elements in an array are greater than a given value.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int i, count = 0;
    int arr[10] = {5, 22, 9, 41, 3, 60, 18, 7, 35, 12};
    int threshold = 20;

    #pragma omp parallel for reduction(+:count)
    for (i = 0; i < 10; i++) {
        if (arr[i] > threshold) {
            count++;
        }
    }

    printf("Numbers greater than %d = %d\n", threshold, count);
    return 0;
}

```

Sample Output

```
Numbers greater than 20 = 4
```

Explanation — Step by Step (Simple Words)

1. The values above 20 are 22, 41, 60, 35 — that's 4 numbers, matching the printed result.
2. count starts at 0 (identity value for a + reduction, same rule as every sum example).
3. Each thread privately increments its own copy of count only when its element passes the condition (arr[i] > threshold).
4. reduction(+:count) then adds all the private counts together to give the final total.

Note: Exactly the same shape as Q9's even/odd counting, just with one condition instead of two.

Q13. Calculate the average of all elements in an array using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int i, sum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int n = 10;
    float average;

    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < n; i++) {
        sum = sum + arr[i];
    }

    average = (float)sum / n;
    printf("Sum = %d, Average = %.2f\n", sum, average);
    return 0;
}

```

Sample Output

```
Sum = 55, Average = 5.50
```

Explanation — Step by Step (Simple Words)

1. This is Q4's exact sum program with ONE extra line added after the parallel region: divide the final sum by n to get the average.
2. Notice the division happens OUTSIDE the parallel region, on the main/master thread only, after all threads have already finished combining sum — it does not need its own pragma.
3. (float)sum forces the division to produce a decimal answer (5.50) instead of doing whole-number integer division (which would incorrectly give 5).
4. $55 / 10 = 5.50$, matching the printed result.

Note: A good reminder: reduction only needs to wrap the part done by many threads — simple follow-up math after the loop stays normal, single-threaded code.

Q14. Calculate the sum of cubes of numbers from 1 to N using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int n = 4;
    long sumCubes = 0;

    #pragma omp parallel for reduction(+:sumCubes)
    for (int i = 1; i <= n; i++) {
        sumCubes += i * i * i;
    }

    printf("Sum of cubes 1..%d = %ld\n", n, sumCubes);
    return 0;
}

```

Sample Output

```
Sum of cubes 1..4 = 100
```

Explanation — Step by Step (Simple Words)

1. $1^3+2^3+3^3+4^3 = 1+8+27+64 = 100$ — matches the printed result.
2. Identical skeleton to Q10 (sum of squares) — only $i*i*i$ replaces $i*i$ inside the loop.
3. `reduction(+:sumCubes)` keeps a private running total per thread, then OpenMP adds them all together at the end.
4. This is the pattern to remember for ANY 'sum of powers from 1 to N' style question.

Note: Last proof of the golden rule: 14 different-looking questions, one single reusable skeleton.

Quick Revision Cheat-Sheet

#pragma omp parallel	Creates a team of threads that run the block together.
#pragma omp parallel for	Splits a for-loop's iterations across threads.
private(var)	Each thread gets its own private copy of a variable.
reduction(+:sum)	Safely combines a variable (e.g. sum) across all threads. Also works with *, max, min in place of +.
omp_get_thread_num()	Returns the calling thread's ID number.

Exam strategy: every question in this guide reuses ONE of two skeletons — (1) plain parallel/parallel-for to split work, or (2) parallel-for + reduction to safely combine a result. Spot which skeleton the question needs, slot in the new logic (factorial, even/odd, squares, cubes, threshold...), and you're done.